Apparent Cross-Language Invariance Reflects Shared Identifiers in Parallel Programs

Naing Oo Lwin[*]
Astrio Labs
nolan.lwin@astriolabs.com

Sree Dhyuti Nimmagadda[*]
Northwestern University
dhyuti@u.northwestern.edu

Ark Dutt[*]
Boston Consulting Group[#]
arksdutt@gmail.com

Michael Ji[*]
University of Waterloo
m2ji@uwaterloo.ca

Randy Lim[*]
New York University
randylim@nyu.edu

Cole Blondin
Algoverse AI Research
cole@algoverseairesearch.org

Archana Vaidheeswaran
Algoverse AI Research
archana@algoverseairesearch.org

Abstract

Cross-language probe transfer is often taken as evidence that a code model represents variable roles independently of programming language. We show that this conclusion depends on whether the readout includes the identifier tokens, which parallel programs systematically preserve. A span-pooled residual probe on Qwen2.5-Coder-1.5B looks flat across the language contrast (-0.012, 95% problem-clustered $[-0.034, 0.020]$), while a model-free surface classifier that masks the identifier loses 0.126 macro-F1 on the same contrast. Excluding the occurrence from the probe readout changes the boundary to -0.060 $[-0.088, -0.030]$, a shift of -0.048 $[-0.088, -0.016]$, in line with context-matched untrained controls. The surface baseline outperforms the context-pooled probe in 14 of 18 matched cells. StarCoder2-7B reproduces the reversal more sharply, moving from -0.014 span-pooled to -0.147 $[-0.186, -0.092]$ occurrence-excluded, with a trained-minus-untrained boundary difference of -0.068 $[-0.114, -0.009]$ that excludes zero. The original invariance is therefore not identified independently of the readout, while the restricted readout still contains training-dependent role information. Code and data are at `github.com/nolanlwin/code-model-interpretability`.

1 Introduction

Linear probes measure whether a label is decodable from a representation, not whether the model uses it, and their performance can reflect probe capacity or information already available in the probe input [Hewitt and Liang, 2019, Voita and Titov, 2020, Ravichander et al., 2021, Elazar et al., 2021, Belinkov, 2022]. Cross-language transfer adds a further interpretation. If a probe fitted on one programming language transfers to another, the decoded feature may appear independent of surface form.

Existing work has recovered code syntax across languages with probe baselines [Hernández López et al., 2026], separated language-specific from language-agnostic embedding components [Utpala et al., 2024], and measured uneven alignment among programming languages on parallel programs [Kargaran et al., 2025]. Within-language suites map what code models encode across many

[*]Equal contribution. [#] Work done prior to joining Boston Consulting Group, which is not connected to this work.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Linguistic Principles for Foundation Models (LP4FM).

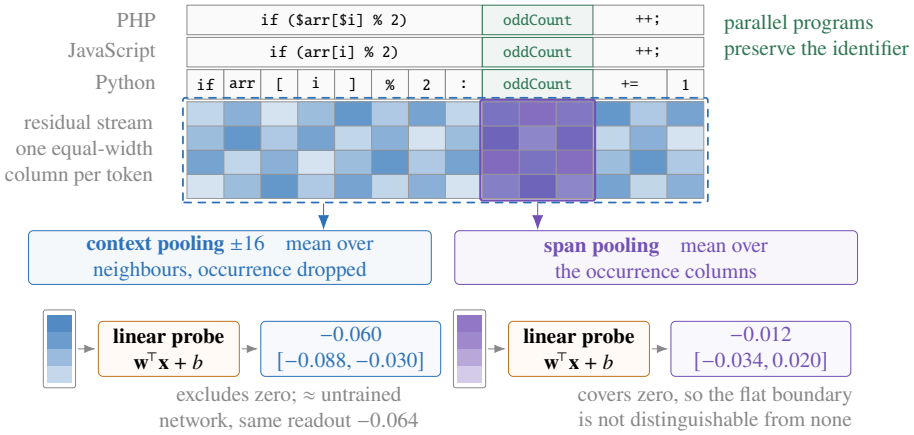


Figure 1: **The cross-language boundary depends on where the probe reads.** Parallel programs preserve the identifier across languages, so a readout that pools the occurrence can transfer for lexical rather than representational reasons. Probes are fitted on the source language and evaluated on the target, and the boundary is mean macro-F1 on Python transfers minus JavaScript↔PHP transfers, with problem-clustered intervals. StarCoder2-7B reproduces the reversal more sharply, -0.014 to -0.147.

probe tasks [Karmakar and Robbes, 2021, Troshin and Chirkova, 2022, Karmakar and Robbes, 2024]. Our contribution is a comparator rather than a new task. These studies establish that code representations contain both shared and language-specific structure, but not a model-free, feature-restricted comparator on the same cross-language prediction task.

We add that comparator for variable roles such as accumulator and index key [Sajaniemi, 2002]. Variable roles describe what a variable does, which makes them a plausible target for language-independent representation. The initial probe result supports that interpretation. It changes when the readout no longer pools the occurrence tokens that carry the identifier, which parallel implementations often preserve [Kargaran et al., 2025]. This ablation localizes the instability to the readout support. It does not, by itself, identify lexical identity as the cause because pooling support and token position change together.

The point generalises past code. Whenever linguistic or symbolic form is manipulated, the instrument must not retain a feature the manipulation preserves, and parallel corpora concentrate such features. In Figure 1 the identifier survives translation, so a readout that pools it reports invariance belonging to the corpus rather than the model. A model-free comparator and a matched readout ablation are inexpensive checks against that.

2 Setup

Corpus. XLCoST [Zhu et al., 2022] provides parallel solutions keyed by problem identifiers that are stable across languages. Python, JavaScript, and PHP are the only three languages with at least 500 shared problems for every pair. For example, C# and Java share only 175 problems (Appendix B). We first restrict each language to the 869 problems present in all three. Role extraction then determines which of those problems contain a usable occurrence for each role, so role-specific coverage still differs across cells. Data splits use a fixed hash of the problem identifier and never separate occurrences from the same problem.

The Python data retain formatting that the JavaScript and PHP data do not. Because this formatting difference coincides with the language contrast, we remove whitespace before constructing character features. Equalizing whitespace on both sides changes each of the three audited iterator transfers by less than 0.001.

Conditions. We evaluate four conditions on matched language pairs.

Table 1: Cross-language role transfer averaged over three roles. *Close* contains JavaScript↔PHP. *Python* contains the four directions to or from Python. Probe floors use the corresponding readout.

	Qwen2.5-Coder-1.5B			StarCoder2-7B		
	close	Python	effect	close	Python	effect
Surface n-gram, name masked	0.929	0.804	-0.126	shared model-free baseline		
Probe, span-pooled (includes occurrence)	0.852	0.840	-0.012	0.877	0.863	-0.014
Probe, occurrence-excluded	0.630	0.569	-0.060	0.734	0.586	-0.147
untrained, occurrence-excluded	0.527	0.464	-0.064	0.552	0.473	-0.080

- **Surface n-gram.** We apply a character n-gram classifier to the context around an occurrence after replacing every instance of the variable's name with a placeholder. The classifier is fitted on the source language and evaluated on the target. We report one fixed masked-window feature set rather than selecting the best feature set separately for each cell.
- **Probe, span-pooled.** Logistic regression on residual activations mean-pooled over the occurrence's own tokens uses a layer chosen on source-language validation data. This readout includes the identifier tokens.
- **Probe, occurrence-excluded.** We use the same probe but pool the 16 tokens on either side of the occurrence while excluding its tokens. Model inputs and weights are unchanged. Only the readout is altered.
- **Untrained.** We evaluate the same architecture and tokenizer with randomly initialized weights under each pooling method. Randomly initialized networks can retain substantial lexical and structural information, so they provide a necessary floor [Zhang and Bowman, 2018, Hewitt and Liang, 2019]. Each probe is compared with the floor using the same readout.

Models are Qwen2.5-Coder-1.5B [Hui et al., 2024] and StarCoder2-7B [Lozhkov et al., 2024]. Their untrained controls use the corresponding architectures and tokenizers with randomly initialised weights. Qwen2.5-Coder is continued-pretrained from Qwen2.5 [Qwen et al., 2024], and we also run the Qwen2.5 base checkpoint as a diagnostic, reported in Appendix A. Reported uncertainty resamples complete problems rather than individual occurrences.

3 Surface transfer tracks the language contrast

The surface classifier reaches 0.929 between JavaScript and PHP without a model or identifier access, then falls to 0.804 on transfers involving Python. The table is an unweighted mean over cells. A separate pooled, problem-clustered estimate is -0.114 with a 95% interval of $[-0.147, -0.081]$. Effects vary across roles. Iterator contributes -0.249, compared with -0.064 for accumulator and index_key together. Iterator labels have the weakest cross-language comparability because Python uses an AST-based extractor while JavaScript and PHP use regular expressions. Some cells contain only ten minority examples. We therefore treat the surface contrast as descriptive evidence for this extracted sample, not as a clean estimate of a language-family effect.

4 The inferred boundary depends on readout support

The span-pooled probe reaches 0.844 overall. Its cross-language boundary is -0.012 with a problem-clustered interval of $[-0.034, 0.020]$. This is the result that would normally be interpreted as language-independent role decodability.

Excluding the occurrence from the readout lowers mean transfer from 0.844 to 0.589 (Figure 3a). The context-matched untrained model reaches 0.485. The boundary changes to -0.060 $[-0.088, -0.030]$. The readout-induced shift is -0.048 $[-0.088, -0.016]$. The original flat boundary is therefore not robust to where activations are read.

The occurrence-excluded untrained model has a -0.064 boundary $[-0.089, -0.039]$. Subtracting this floor gives a trained-minus-untrained boundary of $+0.003$ $[-0.027, 0.036]$. A second weight

initialization yields a -0.035 $[-0.062, -0.008]$ floor boundary with an overall level of 0.497 against the first initialization's 0.485. The two floor levels are similar, while their boundary estimates differ. The trained boundary (-0.060) lies within their range, at its steeper end. Against neither initialization is it reliably flatter. Training still contributes. Trained-minus-untrained transfer is $+0.102$ $[0.065, 0.132]$ close and $+0.106$ $[0.081, 0.127]$ for Python transfers against the first floor, and $+0.109$ $[0.067, 0.145]$ and $+0.084$ $[0.058, 0.107]$ against the second.

A second model. StarCoder2-7B repeats the reversal on the same prediction items, more sharply. Its span-pooled boundary is -0.014 $[-0.036, 0.018]$, as flat as Qwen's. Excluding the occurrence moves it to -0.147 $[-0.186, -0.092]$, a shift of -0.134 $[-0.168, -0.086]$ against Qwen's -0.048. The models differ in how that boundary relates to its floor. StarCoder2's untrained boundary is -0.080 $[-0.110, -0.049]$. Its trained-minus-untrained difference is -0.068 $[-0.114, -0.009]$ and excludes zero, whereas the same contrast was null in Qwen ($+0.003$ $[-0.027, 0.036]$). Readout dependence appears in both tested models. Its relation to the untrained control differs between them.

Identifier exclusion also matters within languages. Mean in-language F1 falls from 0.898 to 0.771 against an untrained 0.623. Direct access to the occurrence accounts for a substantial part of both in-language and cross-language decodability.

The remaining occurrence-excluded margin supports training-dependent role information in this readout, but not a language-independent mechanism. In Qwen the training lift is similar on both sides of the boundary. The surface baseline outperforms the context-pooled probe in 14 of 18 matched cells with problem-clustered intervals below zero for the paired difference (Table 7). This shows surface sufficiency on the same prediction items. It is not a decomposition of the model signal, because the model still processes unmasked source code.

5 Discussion

The central result concerns measurement. A probe that pools over an occurrence can use information concentrated at those tokens, including lexical regularities preserved by parallel programs. A randomly initialized model tests what training contributes under a fixed readout, not whether that readout isolates the intended construct. Occurrence exclusion changes the pooled positions and their information content together. Calling the difference "identifier leakage" would therefore overstate what this experiment identifies.

The model-free condition exposes a different source of evidence. Its input withholds every target-name occurrence, while the probe model sees the original program. The two predictors share labels and test items but not information sets, so their comparison establishes surface sufficiency rather than an isolated representational increment. Model-free baselines are established generally in code probing [Troshin and Chirkova, 2022]. The cited cross-language studies do not fit one on the same prediction items [Hernández López et al., 2026, Utpala et al., 2024, Kargaran et al., 2025]. The present result extends the usual distinction between decodability and model use [Elazar et al., 2021, Belinkov, 2022]. Decodability can also arise from a label correlate exposed directly by the readout.

Reconciling with prior alignment results. Kargaran et al. [2025] find Python the *least* aligned of these three languages on parallel programs, whereas the span-pooled probe changes little across transfers involving Python. The studies measure different quantities, so agreement is not required. Even so, excluding the identifier moves the probe contrast toward their result, which is compatible with a genuine Python boundary without establishing a common mechanism.

Falsifying evidence. The StarCoder2-7B experiment tests a prediction specified in advance. Its close-pair margin over the matched control is $+0.181$, compared with Qwen's $+0.102$, and its boundary difference excludes zero. This limits the deflationary reading. In StarCoder2-7B, the occurrence-excluded boundary is reliably steeper than its untrained counterpart. The measurement claim remains unchanged because the boundary steepened rather than flattened. The interpretation would weaken further if same-readout identifier replacement preserved the flat transfer curve, or if a parallel corpus without shared naming conventions produced the original span-pooled invariance.

Limitations. The study uses two models (1.5B and 7B), three languages, three roles, and one context width. Python supplies the sole broad contrast. Role coverage varies after extraction, so

composition may affect group means. Iterator extractors differ across languages and carry most of the surface effect, and no multilingual hand-audit quantifies extractor error. The untrained controls use two probe seeds rather than five, with two Qwen weight initializations but only one for StarCoder2. The Qwen boundary varies across initializations (-0.064, -0.035), while its level does not (0.485, 0.497). The cross-model comparison therefore rests on one StarCoder2 draw. Occurrence exclusion may retain correlated nearby uses and changes pooling support rather than lexical identity alone, and these results concern decodability, not model use.

References

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. doi: 10.1162/coli_a_00422.

Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the Association for Computational Linguistics*, 9:160–175, 2021. doi: 10.1162/tacl_a_00359.

José Antonio Hernández López, Martin Weyssow, Jesús Sánchez Cuadrado, and Houari Sahraoui. Syntactic multilingual probing of pre-trained language models of code. *Journal of Systems and Software*, 2026. doi: 10.1016/j.jss.2025.112604.

John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1275.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. doi: 10.48550/arXiv.2409.12186.

Amir Hossein Kargaran, Yihong Liu, François Yvon, and Hinrich Schuetze. How programming concepts and neurons are shared in code language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. doi: 10.18653/v1/2025.findings-acl.1379.

Anjan Karmakar and Romain Robbes. What do pre-trained code models know about code? In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1332–1336, Melbourne, Australia, 2021. IEEE. doi: 10.1109/ASE51524.2021.9678927.

Anjan Karmakar and Romain Robbes. INSPECT: Intrinsic and systematic probing evaluation for code transformers. *IEEE Transactions on Software Engineering*, 50(2):220–238, 2024. doi: 10.1109/TSE.2023.3341624.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauss, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Munoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024. doi: 10.48550/arXiv.2402.19173.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi

Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Abhilasha Ravichander, Yonatan Belinkov, and Eduard Hovy. Probing the probing paradigm: Does probing accuracy entail task relevance? In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 3363–3377, Online, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.eacl-main.295.

Jorma Sajaniemi. An empirical analysis of roles of variables in novice-level procedural programs. In *Proceedings IEEE 2002 Symposia on Human Centric Computing Languages and Environments*, pages 37–39, Arlington, VA, USA, 2002. IEEE Computer Society. doi: 10.1109/HCC.2002.1046340.

Sergey Troshin and Nadezhda Chirkova. Probing pretrained models of source codes. In *Proceedings of the Fifth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, pages 371–383, Abu Dhabi, United Arab Emirates (Hybrid), December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.blackboxnlp-1.31. URL `https://aclanthology.org/2022.blackboxnlp-1.31/`.

Saiteja Utpala, Alex Gu, and Pin-Yu Chen. Language agnostic code embeddings. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024. doi: 10.18653/v1/2024.naacl-long.38.

Elena Voita and Ivan Titov. Information-theoretic probing with minimum description length. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. doi: 10.18653/v1/2020.emnlp-main.14.

Kelly W. Zhang and Samuel R. Bowman. Language modeling teaches you more than translation does: Lessons learned through auxiliary syntactic task analysis. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 359–361, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5448.

Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. XLCoST: A benchmark dataset for cross-lingual code intelligence. *arXiv preprint arXiv:2206.08474*, 2022. doi: 10.48550/arXiv.2206.08474.

A Complete transfer results

Table 2: All eighteen transfer cells for Qwen2.5-Coder-1.5B. *Surface* is the strongest masked-context n-gram feature. *Name* uses the variable's name alone. *Probe* is the residual-stream probe. Majority and shuffled-label controls sit near chance throughout. ρ is a descriptive one-instance score step, not a confidence interval or hypothesis test.

role	pair	n	surface	name	probe	major.	shuf.	ρ
accumulator	JavaScript→PHP	1,976	0.951	0.897	0.904	0.361	0.470	0.0005
accumulator	JavaScript→Python	3,341	0.813	0.873	0.875	0.415	0.473	0.0004
accumulator	PHP→JavaScript	1,903	0.954	0.897	0.918	0.372	0.490	0.0005
accumulator	PHP→Python	1,377	0.782	0.824	0.833	0.276	0.439	0.0008
accumulator	Python→JavaScript	3,294	0.785	0.874	0.918	0.370	0.504	0.0003
accumulator	Python→PHP	1,478	0.736	0.845	0.874	0.271	0.466	0.0007
index_key	JavaScript→PHP	1,976	0.940	0.834	0.897	0.384	0.478	0.0005
index_key	JavaScript→Python	3,341	0.859	0.787	0.883	0.330	0.497	0.0003
index_key	PHP→JavaScript	1,903	0.964	0.838	0.880	0.376	0.516	0.0005
index_key	PHP→Python	1,377	0.839	0.782	0.854	0.387	0.515	0.0008
index_key	Python→JavaScript	3,294	0.871	0.822	0.915	0.314	0.470	0.0003
index_key	Python→PHP	1,478	0.858	0.838	0.874	0.419	0.459	0.0008
iterator	JavaScript→PHP	1,976	0.937	0.862	0.906	0.448	0.488	0.0008
iterator	JavaScript→Python	3,341	0.774	0.733	0.939	0.444	0.472	0.0005
iterator	PHP→JavaScript	1,903	0.965	0.873	0.851	0.446	0.478	0.0008
iterator	PHP→Python	1,377	0.576	0.788	0.865	0.428	0.462	0.0010
iterator	Python→JavaScript	3,294	0.790	0.769	0.918	0.466	0.484	0.0007
iterator	Python→PHP	1,478	0.741	0.788	0.902	0.475	0.438	0.0020

Table 3: Problem-clustered intervals for every surface transfer cell. The final row is the Python-pair minus close-pair boundary effect.

role	pair	macro-F1	95% CI	problems
accumulator	JavaScript→PHP	0.925	[0.897, 0.946]	409
accumulator	JavaScript→Python	0.845	[0.812, 0.873]	389
accumulator	PHP→JavaScript	0.953	[0.929, 0.968]	409
accumulator	PHP→Python	0.844	[0.810, 0.873]	378
accumulator	Python→JavaScript	0.853	[0.815, 0.886]	389
accumulator	Python→PHP	0.861	[0.822, 0.891]	378
index_key	JavaScript→PHP	0.875	[0.823, 0.911]	409
index_key	JavaScript→Python	0.878	[0.836, 0.909]	389
index_key	PHP→JavaScript	0.931	[0.902, 0.954]	409
index_key	PHP→Python	0.846	[0.801, 0.882]	378
index_key	Python→JavaScript	0.887	[0.841, 0.919]	389
index_key	Python→PHP	0.881	[0.831, 0.915]	378
iterator	JavaScript→PHP	0.894	[0.808, 0.945]	409
iterator	JavaScript→Python	0.608	[0.554, 0.668]	389
iterator	PHP→JavaScript	0.972	[0.908, 0.994]	409
iterator	PHP→Python	0.564	[0.520, 0.621]	378
iterator	Python→JavaScript	0.848	[0.732, 0.916]	389
iterator	Python→PHP	0.813	[0.705, 0.891]	378
ALL	Python pairs − close pair	-0.114	[-0.147, -0.081]	432

Table 4: Probe transfer per model, averaged within each group. The untrained network shares Qwen2.5-1.5B's architecture and tokenizer with randomly initialised weights. It is the floor a representational claim must clear, and it is itself flat.

model	close pair	Python pairs	all	vs. untrained
Qwen2.5-1.5B	0.899	0.888	0.892	+0.316
Qwen2.5-Coder-1.5B	0.893	0.887	0.889	+0.314
Qwen2.5-1.5B (untrained)	0.590	0.568	0.575	N/A

B Pairwise problem overlap in XLCoST

Table 5: Shared problem identifiers between every pair of XLCoST languages, train split. Matched cross-lingual transfer needs a pair to share enough problems to hold out a test fold. Only the three bold cells clear 500. No pair outside Python/JavaScript/PHP does, including same-family pairs.

	C++	C#	Java	JavaScript	PHP	Python
C++	N/A	115	122	75	17	74
C#	115	N/A	175	75	14	62
Java	122	175	N/A	85	11	66
JavaScript	75	75	85	N/A	**1,529**	**2,953**
PHP	17	14	11	**1,529**	N/A	**1,145**
Python	74	62	66	**2,953**	**1,145**	N/A

C Full masked-probe results

Table 6: Per-cell macro-F1 for the four intersection-sample conditions. *Lift* subtracts the context-matched randomly initialized result from the context-pooled trained result.

role	pair	surface	span	context	random	lift
accumulator	JavaScript→PHP	0.928	0.833	0.551	0.516	+0.035
accumulator	PHP→JavaScript	0.952	0.836	0.640	0.582	+0.058
accumulator	JavaScript→Python	0.835	0.840	0.506	0.513	-0.008
accumulator	PHP→Python	0.831	0.809	0.554	0.424	+0.130
accumulator	Python→JavaScript	0.858	0.866	0.600	0.467	+0.133
accumulator	Python→PHP	0.865	0.854	0.476	0.446	+0.030
index_key	JavaScript→PHP	0.881	0.812	0.696	0.513	+0.183
index_key	PHP→JavaScript	0.935	0.824	0.659	0.568	+0.092
index_key	JavaScript→Python	0.875	0.805	0.661	0.556	+0.105
index_key	PHP→Python	0.858	0.746	0.594	0.370	+0.224
index_key	Python→JavaScript	0.879	0.862	0.672	0.520	+0.151
index_key	Python→PHP	0.878	0.828	0.576	0.328	+0.249
iterator	JavaScript→PHP	0.901	0.950	0.563	0.494	+0.069
iterator	PHP→JavaScript	0.980	0.859	0.668	0.491	+0.177
iterator	JavaScript→Python	0.577	0.836	0.523	0.473	+0.050
iterator	PHP→Python	0.548	0.780	0.517	0.514	+0.003
iterator	Python→JavaScript	0.830	0.966	0.597	0.476	+0.121
iterator	Python→PHP	0.810	0.885	0.556	0.477	+0.080

Table 7: Problem-clustered paired differences between the context-pooled probe and the surface comparator. Negative values favor the surface comparator.

role	pair	Δ	CI low	CI high
accumulator	JavaScript→PHP	-0.315	-0.403	-0.214
accumulator	JavaScript→Python	-0.253	-0.333	-0.178
accumulator	PHP→JavaScript	-0.302	-0.405	-0.211
accumulator	PHP→Python	-0.177	-0.269	-0.083
accumulator	Python→JavaScript	-0.243	-0.335	-0.150
accumulator	Python→PHP	-0.337	-0.443	-0.196
index_key	JavaScript→PHP	-0.069	-0.177	+0.034
index_key	JavaScript→Python	-0.159	-0.306	-0.051
index_key	PHP→JavaScript	-0.131	-0.240	-0.039
index_key	PHP→Python	-0.158	-0.297	-0.023
index_key	Python→JavaScript	-0.114	-0.200	-0.037
index_key	Python→PHP	-0.318	-0.410	-0.239
iterator	JavaScript→PHP	-0.346	-0.507	+0.000
iterator	JavaScript→Python	-0.135	-0.301	-0.031
iterator	PHP→JavaScript	-0.428	-0.508	-0.001
iterator	PHP→Python	-0.074	-0.225	+0.001
iterator	Python→JavaScript	-0.162	-0.392	+0.018
iterator	Python→PHP	-0.190	-0.346	-0.001

Table 8: Problem-clustered uncertainty for the aggregate boundary contrasts. The effect is Python-transfer minus JavaScript↔PHP macro-F1. The difference-in-differences subtracts the untrained boundary from the trained boundary. Intervals resample problem identifiers and are conditional on the committed model runs, including one random weight initialization.

estimand	estimate	95% CI
span-pooled trained boundary	-0.012	[-0.034, +0.020]
occurrence-excluded trained boundary	-0.060	[-0.088, -0.030]
occurrence-excluded untrained boundary	-0.064	[-0.089, -0.039]
trained − untrained boundary	+0.003	[-0.027, +0.036]
training lift, close pair	+0.102	[+0.065, +0.132]
training lift, Python pairs	+0.106	[+0.081, +0.127]
boundary shift after exclusion	-0.048	[-0.088, -0.016]

Table 9: Rolewise occurrence-excluded effects and floor-adjusted transfer. Effects are Python minus close-pair macro-F1. Lift subtracts the matched untrained score. The final column is close-pair lift minus Python-transfer lift. These descriptive role summaries do not carry separate intervals.

role	trained effect	random effect	lift close	lift Python	Δ lift
accumulator	-0.062	-0.087	+0.046	+0.071	-0.025
index_key	-0.052	-0.097	+0.138	+0.182	-0.045
iterator	-0.067	-0.008	+0.123	+0.064	+0.060

Table 10: Per-cell macro-F1 for the StarCoder2-7B replication, on the same prediction items as the Qwen run. *Lift* subtracts the context-matched randomly initialized result from the context-pooled trained result.

role	pair	span	context	random	lift
accumulator	JavaScript→PHP	0.854	0.689	0.658	+0.030
accumulator	PHP→JavaScript	0.859	0.672	0.598	+0.075
accumulator	JavaScript→Python	0.844	0.516	0.516	-0.000
accumulator	PHP→Python	0.821	0.420	0.486	-0.066
accumulator	Python→JavaScript	0.881	0.636	0.561	+0.075
accumulator	Python→PHP	0.884	0.640	0.488	+0.152
index_key	JavaScript→PHP	0.846	0.768	0.573	+0.195
index_key	PHP→JavaScript	0.821	0.780	0.528	+0.252
index_key	JavaScript→Python	0.807	0.637	0.479	+0.158
index_key	PHP→Python	0.778	0.576	0.364	+0.211
index_key	Python→JavaScript	0.870	0.636	0.526	+0.110
index_key	Python→PHP	0.861	0.739	0.334	+0.405
iterator	JavaScript→PHP	0.946	0.767	0.523	+0.244
iterator	PHP→JavaScript	0.934	0.726	0.434	+0.292
iterator	JavaScript→Python	0.880	0.529	0.476	+0.053
iterator	PHP→Python	0.806	0.498	0.464	+0.033
iterator	Python→JavaScript	0.967	0.637	0.475	+0.162
iterator	Python→PHP	0.960	0.571	0.500	+0.071

Table 11: Boundary contrasts for the StarCoder2-7B replication, same estimator and resampling as the Qwen table. Conditional on one random weight initialization.

estimand	estimate	95% CI
span-pooled trained boundary	-0.014	[-0.036, +0.018]
occurrence-excluded trained boundary	-0.147	[-0.186, -0.092]
occurrence-excluded untrained boundary	-0.080	[-0.110, -0.049]
trained − untrained boundary	-0.068	[-0.114, -0.009]
training lift, close pair	+0.181	[+0.118, +0.227]
training lift, Python pairs	+0.114	[+0.086, +0.136]
boundary shift after exclusion	-0.134	[-0.168, -0.086]

Table 12: Qwen2.5-Coder-1.5B boundary contrasts recomputed against the second untrained weight initialization. Trained conditions are unchanged. Only the floor differs from the main table.

estimand	estimate	95% CI
span-pooled trained boundary	-0.012	[-0.034, +0.020]
occurrence-excluded trained boundary	-0.060	[-0.088, -0.030]
occurrence-excluded untrained boundary	-0.035	[-0.062, -0.008]
trained − untrained boundary	-0.025	[-0.061, +0.013]
training lift, close pair	+0.109	[+0.067, +0.145]
training lift, Python pairs	+0.084	[+0.058, +0.107]
boundary shift after exclusion	-0.048	[-0.088, -0.016]

Table 13: Aggregate diagnostics retained in the intersection-sample masked-probe export. Surface counts are test occurrences. Probe counts are shared source–target problems. Surface rows have no in-domain or seed fields. The untrained condition has two probe seeds versus five for the trained conditions.

condition	transfer	in-domain	shuffled source	seeds	cell population
surface window	0.846	N/A	0.476	N/A	2,617–3,138 occ.
span-pooled trained	0.844	0.898	0.468	5	374–406 problems
context-pooled trained	0.589	0.771	0.448	5	374–406 problems
context-pooled untrained	0.485	0.623	0.465	2	374–406 problems

D Renaming and formatting audits

Table 14: Uncapped renaming audit for Python-source transfer. C1, C2, and C4 are the committed identifier-renaming conditions. These samples differ slightly from the frozen main analysis and are reported as a robustness audit rather than pooled with it.

role	condition	target	n_{test}	name	surface	majority
accumulator	C1	JavaScript	12,746	0.396	0.790	0.423
accumulator	C1	PHP	3,517	0.222	0.768	0.417
accumulator	C2	JavaScript	12,746	0.410	0.789	0.423
accumulator	C2	PHP	3,517	0.473	0.786	0.417
accumulator	C4	JavaScript	12,746	0.389	0.793	0.423
accumulator	C4	PHP	3,517	0.565	0.817	0.417
index_key	C1	JavaScript	12,746	0.528	0.845	0.443
index_key	C1	PHP	3,517	0.599	0.880	0.439
index_key	C2	JavaScript	12,746	0.506	0.871	0.443
index_key	C2	PHP	3,517	0.626	0.879	0.439
index_key	C4	JavaScript	12,746	0.523	0.878	0.443
index_key	C4	PHP	3,517	0.509	0.879	0.439
iterator	C1	JavaScript	12,746	0.512	0.662	0.491
iterator	C1	PHP	3,517	0.443	0.650	0.489
iterator	C2	JavaScript	12,746	0.449	0.555	0.491
iterator	C2	PHP	3,517	0.337	0.538	0.489
iterator	C4	JavaScript	12,746	0.468	0.624	0.491
iterator	C4	PHP	3,517	0.431	0.630	0.489

Table 15: Whitespace-normalisation audit for the three decisive iterator surface transfers. Scores are unchanged at the displayed precision.

pair	raw	normalised	Δ
PHP→JavaScript	0.9654	0.9654	+0.0000
PHP→Python	0.5756	0.5756	+0.0000
Python→JavaScript	0.7901	0.7901	+0.0000

E Transfer-mechanism diagnostics

Table 16: Transfer mechanism for the iterator role. *Surviving* is the share of the source classifier's discriminative mass realised in the target. *Flip* is the share of that mass whose sign reverses. Surviving mass is uncorrelated with transfer. Flipped mass predicts it almost exactly. Source-weighted variants are given for robustness.

pair	F1	surviving	flip	agree	flip (src-wt.)
PHP→JavaScript	0.965	0.735	0.072	+0.895	0.072
JavaScript→PHP	0.937	0.698	0.052	+0.945	0.068
Python→JavaScript	0.790	0.862	0.178	+0.695	0.211
JavaScript→Python	0.774	0.757	0.232	+0.811	0.241
Python→PHP	0.741	0.696	0.280	+0.528	0.367
PHP→Python	0.576	0.697	0.366	+0.476	0.346

Table 17: Masking an entire syntactic character class on both sides of the transfer. No class accounts for more than 0.021 of the 0.39 gap, so the realignment is distributed rather than carried by block delimiters or statement terminators.

pair	masked class	macro-F1	Δ
PHP→JavaScript	terminator	0.9496	-0.0159
PHP→JavaScript	brace	0.9645	-0.0010
PHP→JavaScript	bracket	0.9711	+0.0057
PHP→JavaScript	paren	0.9609	-0.0045
PHP→JavaScript	operator	0.9441	-0.0213
PHP→Python	terminator	0.5786	+0.0030
PHP→Python	brace	0.5750	-0.0006
PHP→Python	bracket	0.5733	-0.0023
PHP→Python	paren	0.5947	+0.0191
PHP→Python	operator	0.5801	+0.0045

F Language-geometry controls

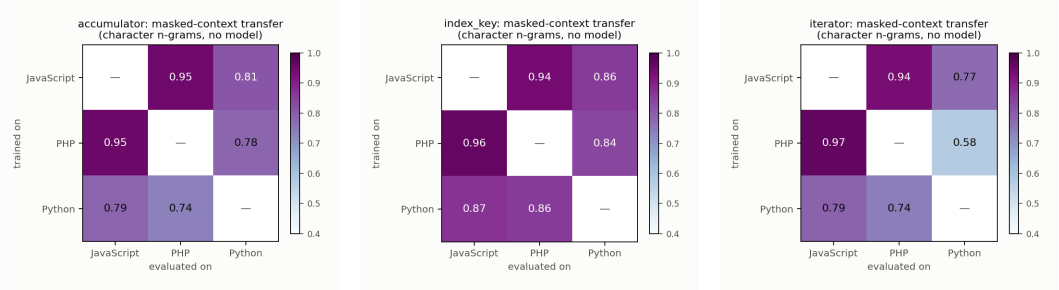
Table 18: Point-biserial correlation between PC1 of last-token residual activations and a static/dynamic typing label, by layer, with PC1's explained-variance ratio. The sign of r is arbitrary (PCA fixes the axis only up to reflection, and each layer is refit independently). Magnitude is what carries meaning.

layer	$\lvert r\rvert$	EVR	layer	$\lvert r\rvert$	EVR
0	0.883	0.481	15	0.894	0.309
1	0.898	0.447	16	0.882	0.303
2	0.914	0.391	17	0.874	0.296
3	0.939	0.373	18	0.873	0.289
4	0.941	0.370	19	0.873	0.299
5	0.862	0.369	20	0.903	0.291
6	0.828	0.361	21	0.886	0.276
7	0.844	0.342	22	0.859	0.265
8	0.896	0.339	23	0.842	0.252
9	0.919	0.333	24	0.812	0.242
10	0.880	0.327	25	0.774	0.253
11	0.893	0.302	26	0.694	0.255
12	0.917	0.292	27	0.732	0.237
13	0.904	0.314	28	0.576	0.242
14	0.898	0.309			

The typing axis is not privileged. Probing the same activations under every alternative 4-versus-3 partition of the seven languages ($\binom{7}{4} = 35$ groupings minus the real one, so 34 controls) gives best-layer macro-F1 0.9959 on average (min 0.9918, max 1.0000), against 1.0000 for the true static/dynamic split. 5 of the 34 controls match or exceed it. Any grouping of these languages is near-perfectly decodable, so the decodability of the typing split is evidence that language identity is encoded, not that type discipline is. At layer 4, where $|r|$ peaks at 0.941, the remaining components carry almost none of the label. PC2 $r = -0.014$, PC3 $r = +0.026$, PC4 $r = +0.171$.

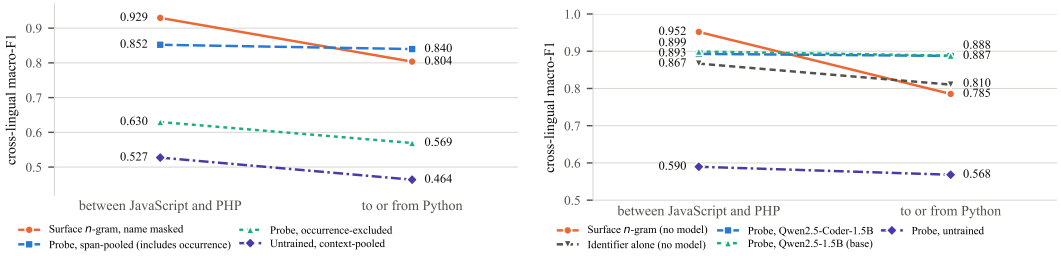
G Complete transfer heatmaps

Reading the heatmaps. Rows are source languages and columns are target languages. These plots retain the complete available transfer matrices, including cells excluded from matched-pair inference because problem overlap is too small.



(a) accumulator (b) index/key (c) iterator

Figure 2: Available transfer matrices per role for Qwen2.5-Coder-1.5B. Cells without adequate matched problem overlap are descriptive only and are not used for the main boundary estimate.



(a) intersection sample, four readout conditions (b) capped sample, span-pooled probes and floor

Figure 3: Cross-language transfer contrasts. Left, the surface comparator, the span-pooled and occurrence-excluded probes, and the context-matched untrained floor on the intersection sample. Right, the span-pooled contrast for both trained models against the architecture-matched random-initialization floor. This capped-sample diagnostic is distinct from the intersection-sample context-pooled analysis.

H Reproducibility

Every figure is regenerated from committed result files, and a regression test recomputes the headline values from those files. Every condition in Table 1, both Qwen floor initializations, the StarCoder2 floor initialization, and the prediction-level boundary analysis are committed per cell. Intervals resample stable problem identifiers once across all cells and conditions. Activation extraction is the only accelerator-dependent step. Probes use logistic regression on residual activations pooled over either the occurrence or the 16 tokens on each side. The surface baseline uses a `char_wb` tf-idf vectorizer with $n \in [2, 4]$ and source-language logistic regression. Store identities record model initialization and pooling so conditions cannot be mixed silently.

I Broader impacts

Positive effects are scientific. A span-pooled probe that looks language-invariant can be read as evidence that a code model represents roles independently of programming language. If such claims later inform multilingual coding tools, a readout-matched comparator reduces that overclaim.

Negative effects follow from the same scores. A flat transfer curve can be treated as invariance if the identifier remains in the readout. Role labels derived from public competitive-programming solutions

could be reused to train name-based heuristics that recover the label from the identifier. No new generative model is released. The source programs were already public.

J Existing assets and licenses

Source programs come from XLCoST under Apache 2.0 [Zhu et al., 2022]. Qwen2.5-Coder-1.5B is used under Apache 2.0. StarCoder2-7B is used under the BigCode OpenRAIL-M license. Each release is cited by name and identifier. Derived role labels inherit Apache 2.0 from XLCoST.

K Code and data

Everything needed to reproduce the paper is at `https://github.com/nolanlwin/code-model-interpretability`. The repository holds the analysis scripts and the committed result files behind every table and figure, together with the structurally labeled XLCoST variable-role data they consume. A dataset card records the fields, the extractors, the splits, and the parent license.